

3. Exercise

Data Science mit Python

Deadline: **06.01.26**, Midnight

Wintersemester 2025/26

Patrick Schäfer, patrick.schaefer@hu-berlin.de

Agenda

- Questions?
- 3. Exercise

Question?

3. Exercise

- The exercise consists of **2 parts**
- 1) Bias and Fairness in Traffic Stops
 - Exploratory Data Analysis
- 2) Pre-Processing Data Sets (Vibe Coding)



Part 1: Bias and Fairness

Bias and Fairness in Traffic Stops

- This dataset contains records of traffic stops conducted by police officers across the United States from 2005 to 2015.
- The data was sourced from the Stanford Open Policing Project
 - A goal is to analyze and improve interactions between police and the public
- The dataset encompasses stops made across all U.S. states during the specified period.
- It has 85845 rows $\times$ 13 columns.

Dataset

- Look at the data
- The dataset contains the following columns:
 - 'stop_date', 'stop_time', 'driver_gender',
 - 'driver_age', 'driver_race', 'violation_raw',
 - 'violation', 'search_conducted', 'search_type',
 - 'stop_outcome', 'is_arrested', 'stop_duration',
 - 'drugs_related_stop'

Dataset

- Familiarise yourself with the data set
- The data are stored in a flat CSV file in tabular form

	stop_date	stop_time	driver_gender	driver_age	driver_race	violation_raw	violation	search_conducted	search_type	stop_outcome	is_arrested
stop_datetime											
2005-01-02 01:55:00	2005-01-02	01:55	M	20	White	Speeding	Speeding	False	NaN	Citation	
2005-01-18 08:15:00	2005-01-18	08:15	M	40	White	Speeding	Speeding	False	NaN	Citation	
2005-01-23 23:15:00	2005-01-23	23:15	M	33	White	Speeding	Speeding	False	NaN	Citation	
2005-02-20 17:15:00	2005-02-20	17:15	M	19	White	Call for Service	Other	False	NaN	Arrest Driver	
2005-03-14 10:00:00	2005-03-14	10:00	F	21	White	Speeding	Speeding	False	NaN	Citation	

Simple Types

`'stop_date': String`

`'stop_time': String`

`'driver_age': Int64`

`'violation_raw': String`

`'search_conducted': Boolean`

`'is_arrested': Boolean`

`'drugs_related_stop': Boolean`

Category Types

`'driver_gender': Category:`
`['F', 'M']`

`'driver_race': Category:`
`['Asian', 'Black', 'Hispanic', 'Other', 'White']`

`'violation': Category:`
`['Speeding', 'Other', 'Equipment', 'Moving violation', 'Registration/plates', 'Seat belt']`

`'stop_outcome': Category`
`['Citation', 'Arrest Driver', 'N/D', 'Warning', 'Arrest Passenger', 'No Action']`

`'stop_duration': Category`
`['0-15 Min', '16-30 Min', '30+ Min']`

List Types

```
'search_type': List
```

Values in:

```
nan, 'Incident to Arrest', 'Protective Frisk', 'Probable Cause', 'Reasonable  
Suspicion', 'Inventory'
```

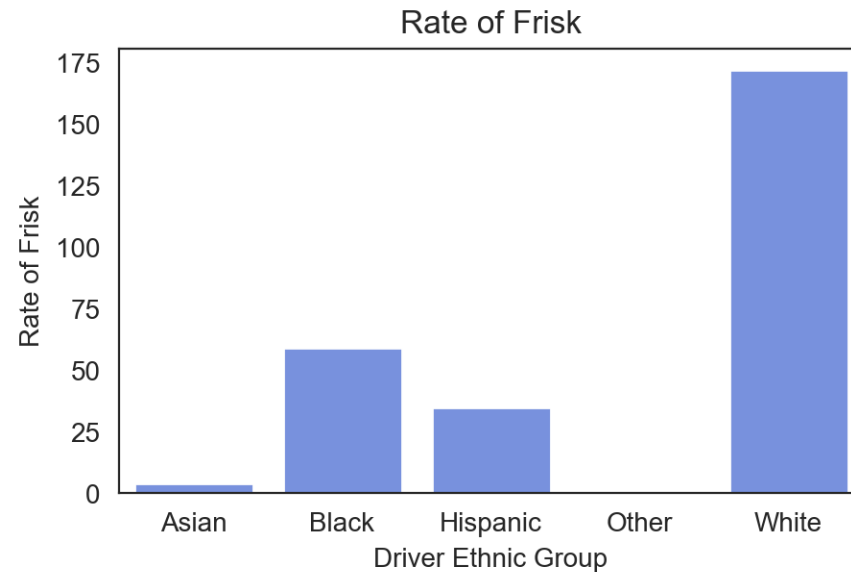
Key Questions

1. Assess the impact of **race, gender, and age** on the frequency of police traffic stops.
2. Evaluate how **gender influences** police behaviour during traffic stops.
3. Examine whether **gender** affects the likelihood of being **frisked** during a search.
4. Compare frisk rates during searches between **males and females**.
5. Compare frisk rates during searches between **Black and White drivers**.
6. Determine which **gender** is more likely to be arrested during a traffic stop.
7. Investigate the impact of **driver age** on the likelihood of being stopped.
8. Analyse whether **stop duration** influences the probability of an arrest.
9. Study how **stop duration** has changed over time to identify evolving policing practices.
10. Use a large language model to suggest additional insightful analyses of the dataset.

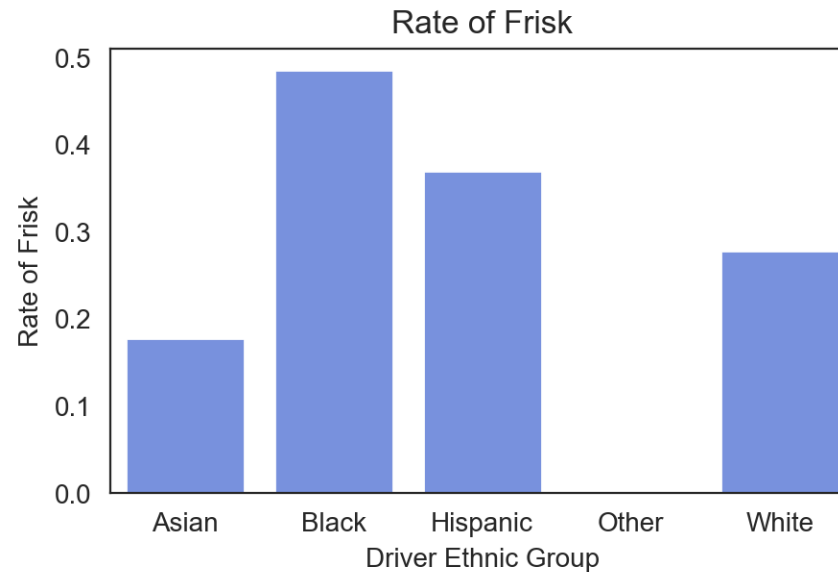
Relative vs Absolute Proportion

count	
driver_race	
White	61758
Black	12152
Hispanic	9448
Asian	2249
Other	238

Total
number of
stops



Absolute Numbers

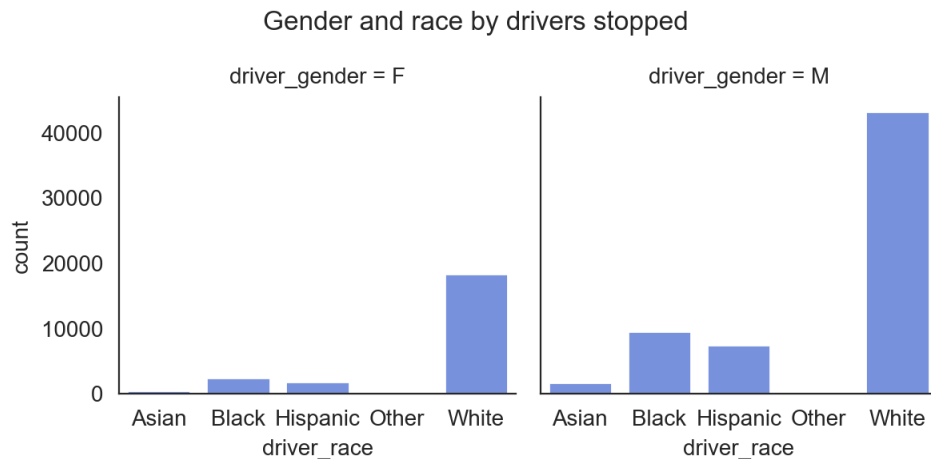


Relative Numbers
(in relation to the total number of stops)

Pay attention to the difference between absolute and relative numbers!

Possible Exemplary Solutions

- Option 1: Plots



- Option 2: Tabular

driver_race		Asian	Black	Hispanic	Other	White
driver_gender						
	F	0.60%	2.99%	2.17%	0.03%	21.47%
	M	2.02%	11.17%	8.83%	0.25%	50.47%

1. **Always add an explanation!**

White males are stopped most often, followed by white females, and black people

Remarks

- Again, **selection, grouping** and **aggregation** are used in the key questions respectively
- Something known from databases:

```
SELECT count(incident_number)
FROM crimes
GROUP BY district
```
- You are supposed to **visualize** and **explain** the results

Jupyter Notebook

- Recommended way to get started
 - Download the starter **Notebook in Moodle**
 - Exercise-3-primer.ipynp
 - Solve the exercise
- We will discuss the primers, next

Part 2. Pre-Processing (Agent)

```
for object to mirror_mod.mirror_object == "MIRROR_X":
    mirror_mod.use_x = True
    mirror_mod.use_y = False
    mirror_mod.use_z = False
    operation == "MIRROR_Y":
    mirror_mod.use_x = False
    mirror_mod.use_y = True
    mirror_mod.use_z = False
    operation == "MIRROR_Z":
    mirror_mod.use_x = False
    mirror_mod.use_y = False
    mirror_mod.use_z = True
    selection at the end -add
    mirror_ob.select= 1
    mirror_ob.select=1
    context.scene.objects.active
    ("Selected" + str(modifier.name))
    mirror_ob.select = 0
    = bpy.context.selected_object
    data.objects[one.name].select
    print("please select exactly one mirror")
-- OPERATOR CLASSES -----
bpy.types.Operator):
    "X mirror to the selected
    object.mirror_mirror_x"
    mirror X"
```

Pre-Processing Agent

- Your task is to perform the steps of a pre-processing agent
- You first need to find a messy dataset from the internet in csv format
 - Kaggle is a great source of inspiration (i.e. Titanic Dataset)
- You are given one Jupyter-Notebooks that guides the general flow
 - `agents_primer.ipynb`

Data Engineering

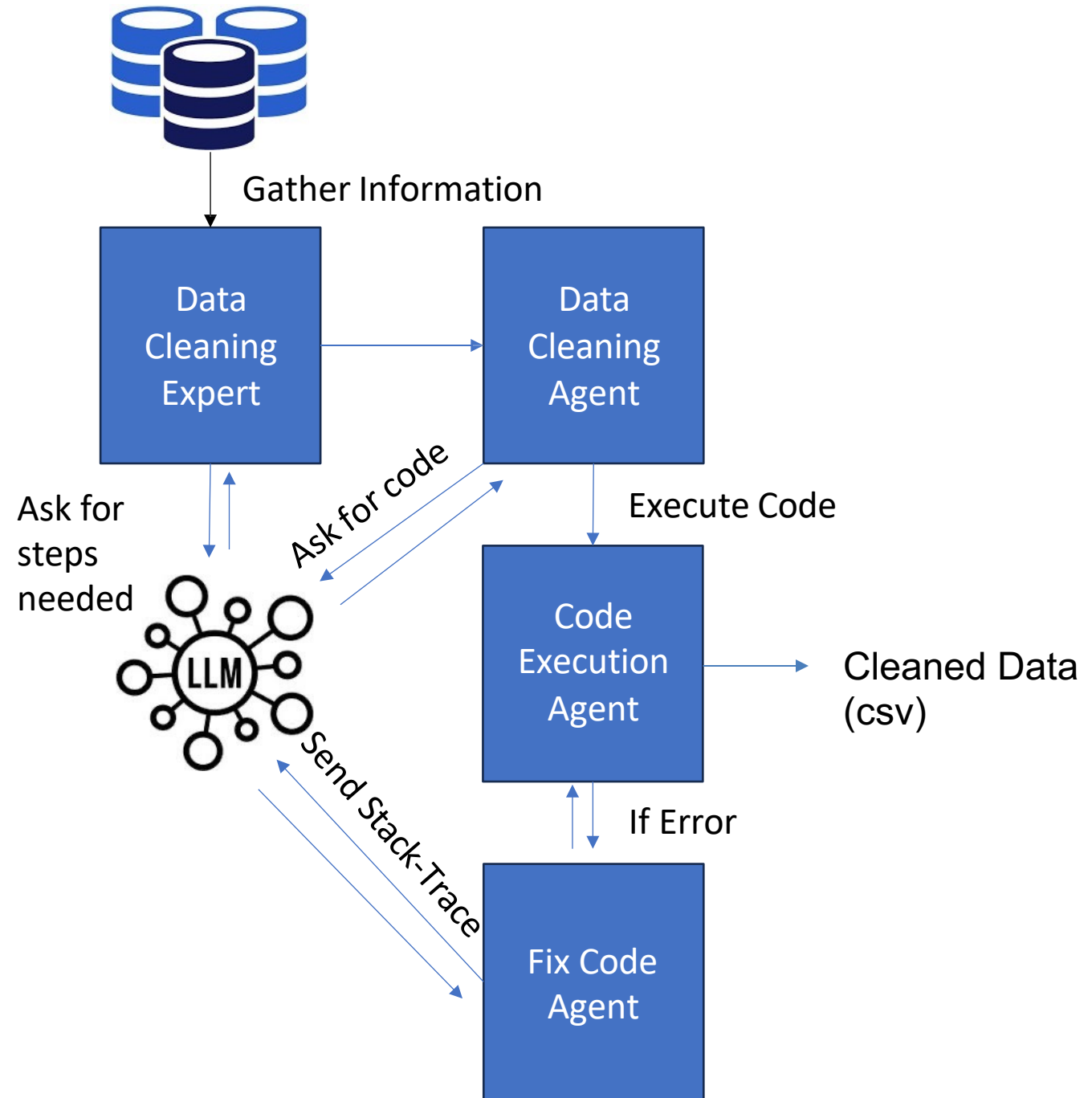
- Garbage In – Garbage Out : Bad input data leads to bad results
- **AI models** depend on the quality and diversity of training data
- Pre-Processing is an important step, to provide cleaner data

Pre-Processing – Traditional Approaches

- **Manual Effort:** Cleaning data requires human effort to identify errors, duplicates, or inconsistencies, which is tedious and prone to oversight
- **Inconsistencies:** Data coming from various sources, each using different formats, leading to inconsistencies and mismatches
- **Time-Intensive:** The larger the dataset, the more time is required to review and clean the data
- **High Error Rates:** Manual cleaning can lead to accidental data loss or inaccuracies, reducing the quality of the final dataset

Agent

- Use a pipeline where each task is handled by an AI agent
- Automates pre-processing
- 4 components
 - Data Cleaning Expert Agent
 - Data Cleaning Agent
 - Code Execution (Agent)
 - Fix Code Agent



Multiple Agents

1. Data Cleaning Expert Agent
 - **Collects dataset characteristics**
 - Prompts LLM for **which steps to perform** based on the data characteristics
2. Data Cleaning Agent
 - Prompts LLM for **code to execute** the required cleaning steps
3. Code Execution Agent
 - **Executes the code** provided by the LLM
 - **We will do this manually**
4. Fix Code Agent
 - In case of an error, provide stack-trace to LLM and re-run code

Data Cleaning Expert

1. Data Cleaning Expert Agent

- **Collects dataset characteristics**
- **Prompts LLM for which steps to perform based on the data characteristics**

```
Dataset Name: athletes_cleaned.csv
Shape: 11498 rows x 9 columns

Column Data Types:
id: int64
nationality: object
sex: object
height: float64
weight: float64
sport: object
gold: int64
silver: int64
bronze: int64

Missing Value Percentage:
id: 0.00%
nationality: 0.00%
sex: 0.00%
height: 0.00%
weight: 0.00%
sport: 0.00%
gold: 0.00%
silver: 0.00%
bronze: 0.00%

Unique Value Counts:
id: 11498
nationality: 287
sex: 2
height: 82
weight: 184
sport: 28
gold: 6
silver: 3
bronze: 3

Data (first 30 rows):
  id nationality sex height weight sport gold silver bronze
0 730841664 ESP male 1.720000 64.000000 athletics 0 0 0
1 512837425 KOR female 1.600000 56.000000 fencing 0 0 0
2 435962083 CAN male 1.980000 79.000000 athletics 0 0 1
3 521841451 HUN male 1.850000 88.000000 taekwondo 0 0 0
4 33922579 NLD male 1.810000 71.000000 cycling 0 0 0
5 173911782 AUS male 1.800000 67.000000 triathlon 0 0 0
6 266227702 USA male 2.050000 98.000000 volleyball 0 0 1
7 382571888 AUS male 1.930000 100.000000 aquatics 0 0 0
8 87689775 ESP female 1.800000 62.000000 athletics 0 0 0
9 997877719 ETH female 1.650000 54.000000 athletics 0 0 0
10 343664681 ETH male 1.700000 63.000000 athletics 0 0 0
11 591319986 BWN male 1.750000 66.000000 athletics 0 0 0
12 280582399 JPN male 1.762574 72.00357 aquatics 0 0 0
13 370658884 USA female 1.610000 49.000000 athletics 0 0 0
14 162792594 USA female 1.780000 68.000000 aquatics 1 1 0
15 521836784 GBR female 1.750000 71.000000 rugby sevens 0 0 0
16 169397772 UZB male 1.610000 57.000000 wrestling 0 0 0
17 256672338 RSA male 1.750000 64.000000 football 0 0 0
18 337369662 NZL female 1.750000 68.000000 football 0 0 0
19 334169879 EGY male 2.100000 88.000000 volleyball 0 0 0
20 215852688 MAR male 1.730000 57.000000 athletics 0 0 0
21 763711885 QAT male 1.850000 88.000000 athletics 0 0 0
22 924585081 SUD male 1.770000 65.000000 athletics 0 0 0
23 578832334 EGY male 1.760000 88.000000 shooting 0 0 0
24 89822258 PAR male 1.900000 72.000000 athletics 0 0 0
25 883161695 ESP male 1.750000 67.000000 athletics 0 0 0
26 189931373 SUD male 1.810000 72.000000 aquatics 0 0 0
27 677622742 ALG male 1.850000 75.000000 football 0 0 0
28 348871891 ALG male 1.860000 72.00357 boxing 0 0 0
29 904080208 ALG male 1.950000 78.000000 football 0 0 0
```

```
You are a Data Cleaning Expert. Given the following information about the data,
recommend a series of numbered steps to take to clean and preprocess it.
The steps should be tailored to the data characteristics and should be helpful
for a data cleaning agent that will be implemented.

General Steps:
Things that should be considered in the data cleaning steps:

• Removing columns if more than 40 percent of the data is missing
• Imputing missing values with the mean of the column if the column is numeric
• Imputing missing values with the mode of the column if the column is categorical
• Converting columns to the correct data type
• Removing duplicate rows
• Removing rows with missing values
• Removing rows with extreme outliers (3X the interquartile range)

Custom Steps:
• Analyze the data to determine if any additional data cleaning steps are needed.
• Recommend steps that are specific to the data provided. Include why these steps are necessary or beneficial.
• If no additional steps are needed, simply state that no additional steps are required.

IMPORTANT:
Make sure to take into account any additional user instructions that may add, remove or modify some of these steps. Include comments in your
code to explain your reasoning for each step. Include comments if something is not done because a user requested. Include comments if someth
ing is done because a user requested.

Below are summaries of all datasets provided:

Dataset Name: athletes_cleaned.csv
Shape: 11498 rows x 9 columns

Column Data Types:
id: int64
nationality: object
sex: object
height: float64
weight: float64
sport: object
gold: int64
silver: int64
bronze: int64

Missing Value Percentage:
id: 0.00%
nationality: 0.00%
sex: 0.00%
height: 0.00%
weight: 0.00%
sport: 0.00%
gold: 0.00%
silver: 0.00%
bronze: 0.00%

Unique Value Counts:
id: 11498
nationality: 287
sex: 2
height: 82
weight: 184
sport: 28
gold: 6
silver: 3
bronze: 3
```

Prompt
Data Science mit Python

LLM – Recommended Steps

- 1. Load and Inspect the Dataset
- 2. Remove Columns With More Than 40 Percent Missing
- 3. Convert Columns to Appropriate Data Types
- 4. Impute Missing Numeric Values (height and weight) Using Mean
- 5. Remove Duplicate Rows
- 6. Remove Rows With Remaining Missing Values
- 7. Detect and Remove Extreme Outliers Using the 3x IQR
- 8. Validate Categorical Consistency
- 9. Analyze the Data for Additional Cleaning Needs
- 9b. Ensure medal counts are non-negative
- 10. Confirm Data Types Again After Cleaning

Numbered Data Cleaning Steps

1. Load and Inspect the Dataset

```
python  
  
df = pd.read_csv("athletes.csv")  
# Inspect shape, dtypes, missing values, unique values to verify metadata
```

This confirms that the file matches the provided summaries before transformation begins.

2. Remove Columns With More Than 40 Percent Missing Data

```
python  
  
# None of the columns exceed 40 percent missing based on metadata  
# Comment to explain: No columns removed because none surpass the threshold.
```

No columns qualify for removal since height has about 2.86 percent missing and weight about 5.71 percent missing.

3. Convert Columns to Appropriate Data Types

```
python  
  
# Convert medal count columns to integer  
df["gold"] = df["gold"].astype(int)  
df["silver"] = df["silver"].astype(int)  
df["bronze"] = df["bronze"].astype(int)  
# Confirm id stays int and categorical columns stay object
```

These medal columns are numeric though stored as object, so converting improves consistency.

4. Impute Missing Numeric Values (height and weight) Using Mean

```
python  
  
df["height"].fillna(df["height"].mean(), inplace=True)  
df["weight"].fillna(df["weight"].mean(), inplace=True)
```

This aligns with the general rules since both columns are numeric.

5. Remove Duplicate Rows

```
python  
  
df.drop_duplicates(inplace=True)
```

The id column has 11538 unique values out of 11543 rows, so some duplicates likely exist. Deduplication ensures row uniqueness.

6. Remove Rows With Remaining Missing Values

Data Cleaning Agent

- **Prompts LLM** for code to execute the required cleaning steps

```
You are a Data Cleaning Agent. Your job is to create a clean_data() function that can be run on the data provided using the following recommended steps.

Recommended Steps:
1. Load and Inspect the Dataset
2. Remove Columns With More Than 40 Percent Missing Data
3. Convert Columns to Appropriate Data Types
4. Impute Missing Numeric Values (height and weight) Using Mean
5. Remove Duplicate Rows
6. Remove Rows With Remaining Missing Values
7. Detect and Remove Extreme Outliers Using the 3x IQR Rule
8. Validate Categorical Consistency
9. Analyze the Data for Additional Cleaning Needs
9b. Ensure medal counts are non-negative
10. Confirm Data Types Again After Cleaning

You can use Pandas, Numpy, and Scikit Learn libraries to clean the data.

Below are summaries of all datasets provided. Use this information about the data to help determine how to clean the data:

Dataset Name: athletes.csv
-----
Shape: 11543 rows x 9 columns

Column Data Types:
id: int64
nationality: object
sex: object
height: float64
weight: float64
sport: object
gold: object
silver: object
bronze: object

Missing Value Percentage:
weight: 5.71%
height: 2.86%
id: 0.00%
nationality: 0.00%
sex: 0.00%
sport: 0.00%
gold: 0.00%
silver: 0.00%
bronze: 0.00%

Unique Value Counts:
id: 11538
nationality: 207
sex: 2
height: 82
weight: 123
sport: 28
gold: 7
silver: 4
bronze: 4

Data (first 30 rows):

```

	id	nationality	sex	height	weight	sport	gold	silver	bronze
0	736041664	ESP	male	1.72	64.0	athletics	0	0	0
1	532037425	KOR	female	1.68	56.0	fencing	0	0	0
2	435962603	CAN	male	1.98	79.0	athletics	0	0	1
3	571041435	MDA	male	1.82	80.0	taekwondo	0	0	0

Prompt

Code Execution Agent

- Runs code locally
- In case there are errors, run fix code agent
- Finally, check, if the data is successfully cleaned by starting the workflow again

```
python Code kopieren

def clean_data(data_raw):
    import pandas as pd
    import numpy as np
    from sklearn.impute import SimpleImputer

    # Make a copy to avoid mutating the original input
    df = data_raw.copy()

    # 1. Load and Inspect the Dataset
    # In this function, the dataset is already loaded and passed in as data_raw.
    # Here we conceptually "inspect" by checking basic structure if needed (no prints t
    # Example (optional for debugging only):
    # print(df.head())
    # print(df.info())

    # 2. Remove Columns With More Than 40 Percent Missing Data
    # Based on the metadata, none of the columns should be removed,
    # but we keep this logic generic so it will work even if the data changes.
    missing_fraction = df.isna().mean()
    cols_to_drop = missing_fraction[missing_fraction > 0.40].index
    # Comment: No columns are expected to be dropped here for athletes.csv,
    # but the code supports the general rule.
    df = df.drop(columns=list(cols_to_drop), errors="ignore")

    # 3. Convert Columns to Appropriate Data Types
    # Ensure "id" is numeric (int). If it is already int64, this is a no-op.
    if "id" in df.columns:
        df["id"] = pd.to_numeric(df["id"], errors="coerce")
        # Any non-numeric ids become NaN and will be handled in the missing data step.

    # Medal columns are stored as object but represent integer counts.
    # Convert them to numeric, coercing invalid entries to NaN for later cleaning.
    for col in ["gold", "silver", "bronze"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    # 4. Impute Missing Numeric Values (height and weight) Using Mean
    # We follow the recommendation to use the mean for numeric imputation.
    # Use SimpleImputer from scikit-learn and ravel the result as required.
    imputer = SimpleImputer(strategy="mean")

    for col in ["height", "weight"]:
        if col in df.columns:
            # fit_transform returns a 2D array, but a Series column is 1D,
            # so we ravel the array before assignment (best practice).
            df[col] = imputer.fit_transform(df[[col]]).ravel()

    # 5. Remove Duplicate Rows
    # Drops fully duplicated rows to avoid repeated records.
    df = df.drop_duplicates()

    # 6. Remove Rows With Remaining Missing Values
```

Submission Deadline: Tue, 06.01.26

- The exercise is considered to have been **successfully solved**, if
 - Part 1: An answer backed by evidence has been given for each of the **6 key questions**
 - Part 2: Pre-processing workflow **was successfully executed and data is cleaned**
- **Deliverables:**
 - **Part 1: EDA (2 Files)**
 - a) **Executed notebook**
 - b) A **html export** of the **executed notebook**
 - **Part 2: Agent (6 Files):**
 - a) **Executed Notebook and HTML export**
 - b) **Original and cleaned CSV file**
 - c) **Summary Statistics before and after cleaning**

Bonusblatt

Questions?